Project 38 — Parallel LLM Inference with a Fault-Tolerant Pipeline

Implementation plan, start to end · Student: MD TAUFIQUE HUSSAIN · Written 8 Nov 2026

> **Goal in one sentence:** cut GPT-2 small (12 layers) into two halves, run each half in its own process, connect them with gRPC, and prove the pair writes **exactly the same tokens** as the normal model, with many users at once and even when a worker is killed mid-sentence.

Related files: `plan.md` (detailed reasoning), `Project_overview.pdf` (proposal), `Project38_Concept_Guide.pdf`, `learning_book/` (my chapter-by-chapter notes).

---

1. How to use this plan

Every phase has the same four parts:

1. **Learn** — the book chapter to read first (so no code is written blind).
2. **Build** — small tasks, in order.
3. **Test** — a command that must pass.
4. **Done when** — the exit gate. Do not start the next phase until it is met.

Golden rule: never build the distributed version before the single-process version is proven correct. When output differs, you then know whether the bug is in the *model* or in the *network*.

2. The big picture

```
 Phase 0  Setup                          ┐
Phase 1  Single-process reference        │  MAKE IT CORRECT
Phase 2  In-process split                ┘
Phase 3  Two processes over gRPC         ┐
Phase 4  Concurrency + ordering          │  MAKE IT DISTRIBUTED
Phase 5  Bounded queues, backpressure    ┘
Phase 6  Failure detection + recovery       MAKE IT SURVIVE   (most important)
Phase 7  Micro-batching (optional)          MAKE IT FAST
Phase 8  Benchmarks                      ┐
Phase 9  Report, README, demo            ┘  PROVE IT
```

Architecture you are building:

```
 Client ──prompt──► Coordinator ──token IDs, step, epoch──► Worker A (embeddings +
blocks 0–5, KV cache)
        ◄──tokens──   (brain)                                    │ activations:
768 floats per token
                          ▲                                      ▼
                          └──────────── next token ──────────  Worker B (blocks 6–11 +
LM head, KV cache)
```

3. Timeline (about 8 weeks, adjust to your deadline)

Week	Phase	Days	Risk
1	0 Setup + 1 Reference	2–3 + 4–5	Low
2	2 In-process split	2–3	Low
3	3 gRPC pipeline	5–6	Medium
4	4 Concurrency + ordering	5–6	Medium
5	5 Backpressure	3–4	Low
5–6	**6 Failure + recovery**	7–8	**High**
7	7 Micro-batching (stretch) + 8 Benchmarks	5–6	Medium
8	9 Report + demo	4–5	Low

Protect Phases 6 and 8. If time runs short, **drop Phase 7 first**.

Phase 0 — Setup (2–3 days)

Learn: Ch. 1 (what a distributed system is).

Build

- ☐ Create the repo with the layout in section 5.
- ☐ Python 3.11 virtual environment:

```
        python3.11 -m venv .venv && source .venv/bin/activate
        pip install torch transformers grpcio grpcio-tools pytest pandas
matplotlib psutil
        pip freeze > requirements.txt     # pin versions on day 1 and commit
```

- ☐ Download GPT-2 once and save it locally so workers load from disk:

```
        from transformers import GPT2LMHeadModel, GPT2TokenizerFast
        GPT2LMHeadModel.from_pretrained("gpt2").save_pretrained("models/gpt2")
        GPT2TokenizerFast.from_pretrained("gpt2").save_pretrained("models/gpt2")
```

- ☐ Write `data/prompts.json` with 20 prompts of varied length and topic.

Done when: the model loads from `models/gpt2` offline and `requirements.txt` is committed.

Phase 1 — Single-process reference with your own KV cache (4–5 days)

Learn: Ch. 2 (how an LLM generates text), Ch. 3 (attention and KV cache).

Build `src/model_core.py`

- [ ] `embed(token_ids, positions)` = token embedding + position embedding.
- [ ] `run_block(block, h, cache)` = `ln_1 → attention (using and extending the cache) → +residual → ln_2 → mlp → +residual`.
- [ ] `lm_head(h)` = `ln_f` then multiply by the tied embedding matrix.
- [ ] `generate_reference(prompt_ids, max_new_tokens=50)`: one **prefill** over the whole prompt, then a **decode** loop with `argmax` (greedy).
- [ ] Run 20 prompts × 50 tokens and save to `data/reference_outputs.json`. **This is the ground truth for the whole project.**

Test: `pytest tests/test_reference.py` — your loop equals `model.generate(do_sample=False)` for all 20 prompts.

Common bugs (check in this order): positions not offset during decode → causal mask in prefill → forgot `ln_f` → dtype not float32.

Done when: 20/20 prompts match.

Phase 2 — In-process split (2–3 days)

Learn: Ch. 4 (model and pipeline parallelism), Ch. 5 (activations, serialization).

Build `src/stages.py`, `src/serialization.py`

- [ ] `Stage1`: owns `wte, wpe, blocks[0:6]` and `kv_cache[request_id]`. `forward(request_id, token_ids, positions) → hidden`.
- [ ] `Stage2`: owns `blocks[6:12], ln_f, lm_head` and its own cache. `forward(request_id, hidden) → next_token`.
- [ ] `release(request_id)` on both.
- [ ] Make the split point configurable (`--split 6`).
- [ ] `to_bytes(tensor)` / `from_bytes(...)` with a round-trip test.

Done when: in one Python process, `Stage1 → bytes → Stage2` reproduces all 20 references exactly, including two requests interleaved step by step.

Phase 3 — Two processes over gRPC (5–6 days)

Learn: Ch. 5 (RPC), Ch. 6 (requests, steps, ordering).

Build

- [ ] `proto/pipeline.proto` (section 6) and generate code:

```
        python -m grpc_tools.protoc -I proto --python_out=src --
grpc_python_out=src proto/pipeline.proto
```

- [ ] `worker.py --stage A|B --port … --epoch …` serving `Forward`, `Release`, `Heartbeat`.
- [ ] Data path: **coordinator → A → B → back through A to coordinator**, so activations travel only between A and B.
- [ ] `coordinator.py`: assigns UUIDs, runs prefill then decode steps, keeps `prompt_ids` and `generated_ids` (needed for replay later), calls `Release` at the end.
- [ ] `client.py`: prints tokens as they stream back (server-streaming RPC).
- [ ] `launcher.py`: starts both workers with `subprocess.Popen`, waits for heartbeats, then starts the coordinator.
- [ ] **Deadline on every RPC** and **gRPC automatic retries turned off**, so every retry passes through your own step and epoch logic.
- [ ] Log bytes per message (decode ≈ 3 KB per step, a 100-token prefill ≈ 300 KB).

Test: `pytest tests/test_pipeline.py` (Test 2) — all 20 prompts identical to the reference. Then try once across two laptops on the same LAN.

Phase 4 — Concurrency, ordering, duplicates (5–6 days)

Learn: Ch. 6 (ordering), Ch. 7 (concurrency, pipelining), Ch. 12 (idempotency).

Build

- [ ] One `asyncio` task (or thread) per request, so A works on request 2 while B works on request 1.
- [ ] Worker rule per `request_id`:

Incoming	Action
`epoch < my_epoch`	discard, reply `STALE_EPOCH`
`step == expected`	process, store result, `expected += 1`
`step < expected`	duplicate: return stored result, do not touch cache
`step > expected`	reply `OUT_OF_ORDER`, coordinator resends

- [ ] `last_result[request_id]` so duplicates are answered without recomputing.
- [ ] Coordinator delivers each `(request_id, step)` to the client **exactly once**.
- [ ] Per-request lock; sensible `torch.set_num_threads()` per worker.
- [ ] Idle-timeout cleanup of abandoned caches.
- [ ] Test-only fault hooks that resend an old step or old epoch.

Test: Test 3 (10 concurrent prompts, each equals its own reference) and Test 6 (injected duplicates and stale epochs all discarded).

Phase 5 — Bounded queues and backpressure (3–4 days)

Learn: Ch. 7 (bubbles), Ch. 9 (backpressure).

Build

- ☐ Worker input queue, capacity 8; full → reply `BUSY` at once.
- ☐ Coordinator on `BUSY`: exponential backoff 10, 20, 40 ms… (capped), then retry.
- ☐ `max_in_flight` admission limit; extra clients get `RESOURCE_EXHAUSTED`.
- ☐ `--delay-ms` on Worker B to simulate a slow stage.
- ☐ Log worker memory (`psutil` RSS) over time.

Test: Test 7 — flood beyond capacity; clients get BUSY or rejection, nothing crashes, memory stays flat.

Phase 6 — Failure detection and recovery (7–8 days, the heart of the project)

Learn: Ch. 8 (timeouts, failure detection), Ch. 9–11 (restart, replay, epochs).

6a Detection

- ☐ Heartbeat to each worker every 500 ms. 3 misses in a row, or a `Forward` past its deadline → declared failed. Log the detection time.

6b Recovery procedure

1. **Fence:** pause all request loops.
2. **Restart:** launcher starts a fresh process with `epoch = old + 1`; it loads weights from disk; wait for its heartbeat.
3. **Epoch update:** coordinator records the new epoch; every later message carries it.
4. **Replay:** for each unfinished request send `REPLAY` with `prompt_ids + generated_ids`. Each worker clears the old cache for it, runs one prefill, and sets `expected_step`. Worker B's reply is the next token the client has not seen.
5. **Resume** decode loops. Delivered tokens are never resent.
6. Late old-epoch messages are discarded by the Phase 4 rule.

6c Tooling

- ☐ `kill_at_step.py --worker B --step 17` (SIGKILL).
- ☐ Randomised crash loop: random worker, random step, assert equality every time.

Watch out for: a step that was *in flight* during the crash (not delivered unless the token came back); requests finishing or arriving during recovery; replay needing a longer deadline.

Test: Tests 4 and 5 — kill B and kill A at early, middle and late steps, many random runs.

Done when: output is identical, no token is delivered twice, in every run.

Phase 7 — Micro-batching (stretch, 3–4 days)

- [ ] Worker gathers up to 4 ready decode steps (or waits a few ms), stacks them, uses left-padding plus an attention mask and per-request positions, then splits outputs back per request.
- [ ] Tokens must equal the reference; compare logits within 1e-4 and document any rare argmax flip.

Phase 8 — Benchmarks (5–6 days)

Log per token: `request_id, step, epoch, t_submit, t_token, bytes_A→B, stage_A_ms, stage_B_ms` to CSV; generate every graph from the CSVs by script. Repeat each run ≥ 5 times; report mean and p95.

#	Experiment	Vary	Plot
E1	Baseline vs pipeline	1 process / 2 processes / 2 laptops	TTFT, per-token latency
E2	Concurrency	1, 2, 4, 8 clients	Throughput, p95 latency
E3	Micro-batching	on/off	Throughput, idle time
E4	Communication	prompt 16, 64, 256	Bytes: prefill vs decode
E5	Recovery	kill A/B at early/mid/late	Detect + restart + replay time
E6	Overload	queue 4, 8, 16	Rejections, latency, memory

Expected story: pipeline is slower for one request; throughput rises then saturates with concurrency; decode bytes are tiny next to prefill/replay bytes; recovery cost grows with how late the crash is; memory stays bounded.

Phase 9 — Report, README, demo (4–5 days)

- [ ] README: install, download model, run `launcher.py`, run tests, run benchmarks.
- [ ] Report: architecture, message format, ordering rules, recovery timeline diagram, failure assumptions (crash-stop, reliable coordinator), graphs, overhead analysis, limitations (coordinator is a single point of failure), future work.
- [ ] Rehearsed demo: (1) same text single vs pipeline, (2) five clients at once, (3) `kill -9` Worker B mid-sentence and watch *detected → restarted (epoch 2) → replayed → identical text*, (4) flood the system and show it stays up.

---

4. Test matrix

#	Test	Passes when	Phase
1	20 prompts × 50 tokens, single process	saved as ground truth	1
2	Same prompts through the pipeline, one at a time	identical tokens	3
3	10 prompts concurrently	each equals its own reference	4
4	Kill Worker B at a random step	identical, no token twice	6
5	Kill Worker A at a random step	identical, no token twice	6
6	Inject duplicates and old-epoch messages	all discarded	4
7	Flood beyond queue capacity	BUSY, no crash, memory bounded	5

5. Repo layout

```
project38/
├── proto/pipeline.proto
├── src/  model_core.py  stages.py  serialization.py  worker.py
│         coordinator.py  client.py  launcher.py  metrics.py
├── tests/ test_reference.py  test_pipeline.py  test_concurrency.py
│          test_crash.py  test_duplicates.py  test_overload.py
├── bench/            # E1–E6 scripts, CSVs, plot scripts
├── data/             # prompts.json, reference_outputs.json
├── models/gpt2/      # saved weights
├── requirements.txt  # pinned
└── README.md
```

6. Message format (protobuf sketch)

```
         enum Kind   { PREFILL = 0; DECODE = 1; REPLAY = 2; RELEASE = 3; HEARTBEAT
= 4; }
        enum Status { OK = 0; BUSY = 1; OUT_OF_ORDER = 2; STALE_EPOCH = 3;
DUPLICATE = 4; }

        message Tensor { bytes data = 1; repeated int32 shape = 2; string dtype =
3; }

        message StageRequest {
          string request_id = 1;
          int32  step       = 2;      // 0 = prefill, 1.. = decode
          int64  epoch      = 3;      // incarnation of the receiving worker
          Kind   kind       = 4;
          repeated int32 token_ids = 5;
          repeated int32 positions = 6;
          Tensor hidden     = 7;
        }
        message StageReply { Status status = 1; int32 next_token = 2; int32 step =
3; int64 epoch = 4; }

        service Stage {
          rpc Forward(StageRequest)   returns (StageReply);
          rpc Release(StageRequest)   returns (StageReply);
          rpc Heartbeat(StageRequest) returns (StageReply);
        }
```

7. Top risks

Risk	Mitigation
Hugging Face cache format changes	Own KV cache in `model_core.py`; pinned versions
Output "almost" matches	Check positions → mask → `ln_f` → dtype
Race conditions in recovery	Pause loops during recovery; per-request locks; many randomised crash runs
gRPC hidden retries	Disable; all retries go through step/epoch logic
Two workers fight for CPU on one laptop	`torch.set_num_threads` per worker; mention in report
Schedule slips	Drop Phase 7 first; never cut 6 or 8

8. Your next three actions

1. Phase 0: repo, virtual environment, pinned `requirements.txt`, GPT-2 saved locally.
2. Write `data/prompts.json` (20 prompts).

3. Read Chapters 2–3 with me, then start `model_core.py` and match `generate()` on one prompt.